



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: PROJECT EVALUATION (24CS2235F)

A.Y. 2026 - 2027

TITLE OF THE PROJECT		AI-Driven Personalized Healthcare Decision Support and Remote Patient Monitoring Platform	
S. No.	University ID	Name	
1	2420030200	S.Sai Charan	
2	2420090056	D.Sree Aditya	
3	2420030515	K.Vishnu vardhan	
4	2420030359	Roopesh Chalasani	
Name of the Guide		B.Dwarakanath	

Abstract

The Code Optimization Simulator is a Compiler Design project that demonstrates how a compiler improves program code before it is converted into machine-level instructions. The main purpose of the project is to identify unnecessary, repeated, and inefficient operations in intermediate code and replace them with a simpler and more efficient form. The project focuses on important compiler concepts such as Basic Blocks, DAG (Directed Acyclic Graph) Representation, Dead Code Elimination, Constant Propagation, and Common Subexpression Elimination. The simulator takes simple intermediate code as input from the user. First, the input code is analyzed and divided into Basic Blocks, which are small groups of statements that execute together without unnecessary jumps in between. After identifying the basic blocks, the system uses DAG representation to understand the relationships between variables and expressions and to identify repeated calculations. The project then performs different optimization techniques. Dead Code Elimination removes statements that are not required or whose results are never used. For example, if a variable is assigned a value but is never used later, that statement can be removed. Constant Propagation replaces a variable with its known constant value wherever possible. Common Subexpression Elimination identifies expressions that are calculated more than once and reuses the previously calculated result instead of performing the same calculation again. After applying these optimization techniques, the simulator generates the optimized intermediate code. The system displays the original code and optimized code so that users can easily compare the changes and understand how optimization reduces unnecessary operations. The project also helps demonstrate how the optimization phase works inside a compiler and how optimized code can improve execution efficiency and reduce the number of instructions. Overall, the Code Optimization Simulator provides a simple and interactive way to understand compiler optimization techniques. It helps students learn how a compiler analyzes intermediate code, detects unnecessary computations, and produces a more efficient version of the program without changing its intended output.

Signature of the Guide

HOD